

Trees & Linear Structures Cheatsheet

Lessons: [42](#) — [Trees](#) · [50](#) — [Stacks, Queues, Deques & Linked Lists](#)

Examples: [42](#) · [50](#)

Covers: recursive pointer structs, traversals, BSTs, tries, slice-as-stack/queue, linked lists, LRU

Legend: `[*]` = API the lessons have not covered yet

THE SLICE IS YOUR STACK

```
var s []int           the zero value is a ready, empty stack
s = append(s, v)      push – 0(1) amortized
v := s[len(s)-1]      peek
s = s[:len(s)-1]      pop
len(s) == 0           empty check
(check the length before peeking – indexing an empty slice panics)
```

THE SLICE AS A QUEUE (and its leak)

```
q = append(q, v)      enqueue
v := q[0]; q = q[1:]  dequeue – 0(1), BUT the head is never freed
THE LEAK              q[1:] keeps the whole backing array alive;
                     for pointer elements, nothing is ever collected
fix 1: head index     keep head int; q[head]; head++; compact when head > len/2
fix 2: ring buffer    a fixed array + head/tail/count – no allocation at all
fix 3: copy down      copy(q, q[1:]); q = q[:len(q)-1] – 0(n) per dequeue
fix 4: nil the slot   q[0] = nil before re-slicing (pointer elements only)
```

GENERIC STACK & QUEUE

```
type Stack[T any] struct { items []T }
func (s *Stack[T]) Push(v T)      { s.items = append(s.items, v) }
func (s *Stack[T]) Pop() (T, bool) {
    var zero T
    if len(s.items) == 0 { return zero, false }
    v := s.items[len(s.items)-1]
    s.items = s.items[:len(s.items)-1]
    return v, true
}
(the comma-ok return is the Go way to say "empty" – no panics, no sentinel values)
```

DEQUE & container/list

```
deque                push/pop at BOTH ends; a ring buffer, or a slice with
                     head and tail indices
container/list       [*] a doubly linked list in the stdlib
    l := list.New(); l.PushBack(v); l.PushFront(v)
    l.Front() / l.Back() / l.Remove(e) / e.Value / e.Next()
    it holds `any` – every access is a type assertion
container/ring       [*] a circular list, for fixed-size rotation
(a slice beats container/list for almost everything – cache locality wins)
```

LINKED LISTS

```
type Node struct { Val int; Next *Node }    the recursive pointer struct
nil IS the empty list                      no sentinel needed; that's the base case
traverse                                  for n := head; n != nil; n = n.Next { ... }
prepend                                  n := &Node{Val: v, Next: head}; head = n    0(1)
append                                   walk to the tail, or keep a tail pointer
reverse                                  prev, cur := (*Node)(nil), head
                                          for cur != nil { next := cur.Next; cur.Next = prev;
                                          prev, cur = cur, next }; return prev
middle                                  slow/fast pointers: fast moves 2, slow moves 1
cycle detection (Floyd)                  slow/fast; if they ever meet, there's a cycle
                                          then reset slow to head, step both by 1 -> the entry
merge two sorted                         a dummy head node makes the code half as long
dummy head                              &Node{} in front removes every "is this the first?" case
doubly linked                            Prev *Node too – 0(1) removal given the node
(use one when you need 0(1) splice/remove with a held reference; otherwise a slice)
```

BINARY TREES

```
type Tree struct { Val int; Left, Right *Tree }
nil is the empty tree                    every recursion's base case: if t == nil { return }
height                                  1 + max(height(L), height(R)); nil -> 0
size                                    1 + size(L) + size(R)
depth-first recursion                    the natural shape; watch the stack on deep trees
```

TRAVERSALS (memorize the three lines)

```
pre-order      visit, Left, Right      – copy/serialize a tree
in-order       Left, visit, Right       – a BST comes out SORTED
post-order     Left, Right, visit       – free/aggregate children first
level-order (BFS) a QUEUE, not recursion:
  q := []*Tree{root}
  for len(q) > 0 {
    n := q[0]; q = q[1:]
    if n.Left != nil { q = append(q, n.Left) }
    if n.Right != nil { q = append(q, n.Right) }
  }
level by level capture len(q) BEFORE the inner loop – that's one level
iterative DFS  an explicit stack; push Right before Left for pre-order
```

BINARY SEARCH TREES

the invariant	every Left < node < every Right, RECURSIVELY
search	$O(h)$: go left if smaller, right if larger
insert	walk to a nil child and hang the node there (return the subtree from the recursive call – the `t.Left = insert(t.Left, v)` idiom keeps it simple)
delete (three cases)	no children -> nil one child -> that child two children-> replace with the in-order SUCCESSOR (leftmost of the right subtree), then delete the successor
validate	in-order must be strictly increasing, OR pass (min, max) bounds down the recursion – NOT just comparing each node with its parent
$h = \log n$ only if balanced	sorted input builds a linked list; that's why AVL / red-black / B-trees exist

OTHER TREE SHAPES

generic BST[T cmp.Ordered]	the same code with a type parameter
serialize / deserialize	pre-order with an explicit "#" for nil, or level-order
trie (prefix tree)	type Trie struct { children map[rune]*Trie; end bool } insert/search/startsWith are all $O(\text{len}(\text{word}))$
expression tree	leaves are values, internal nodes are operators; evaluate with post-order
n-ary tree	Children []*Node – the filesystem/DOM shape

APPLIED IDIOMS

balanced brackets	push openers, pop and match on closers, empty at the end
RPN evaluation	push operands, pop two on an operator
min-stack	a second stack holding the minimum so far -> $O(1)$ Min
queue from two stacks	in-stack and out-stack; move only when out is empty
monotonic stack	keep it increasing/decreasing -> next-greater-element, largest rectangle, daily temperatures – all $O(n)$
monotonic deque	the sliding-window maximum in $O(n)$
LRU cache	map[K]*list.Element + a doubly linked list get: move to front; put: evict from the back – $O(1)$ both
buffered channel	Go's built-in CONCURRENT queue: make(chan T, n) – use it instead of a mutex-wrapped slice

COMPLEXITY TABLE

slice index	$O(1)$
slice append	$O(1)$ amortized ($O(n)$ on the growth copy)
slice insert/delete middle	$O(n)$
linked list prepend	$O(1)$
linked list search	$O(n)$
linked list remove (w/ node)	$O(1)$
BST search/insert/delete	$O(h)$: $O(\log n)$ balanced, $O(n)$ degenerate
tree traversal	$O(n)$ time, $O(h)$ stack
trie op	$O(\text{len}(\text{key}))$
LRU get/put	$O(1)$

TRAPS & MEMORIZE

the slice-queue leak	<code>q = q[1:]</code> never frees the head
peeking an empty slice	panic – check <code>len</code> first
forgetting to reassign append	<code>s = append(s, v)</code> , always
recursive delete without reassignment	<code>t.Left = delete(t.Left, v)</code>
BST validation vs the parent	a node can be > its parent and still break the tree
deep recursion	Go's stack grows, but 1e6 frames is still a problem
container/list's <code>`any`</code>	a type assertion at every read; prefer a generic slice
comparing struct pointers	<code>==</code> compares addresses, not contents
building a BST from sorted data	it degenerates into a linked list
losing the head pointer	walking with head instead of a cursor variable
nil map in a trie node	make it before the first child insert